



python

Pr. Houda BENHAR
Année universitaire 2023-2024

PLAN

- Les strings
- Les listes
- Les tuples
- Les dictionnaires



STRINGS

- Une chaîne est une suite (séquence) de caractères. Vous pouvez accéder aux caractères un par un avec l'opérateur crochet:

```
language = "python"  
letter = language[1]
```

- La deuxième instruction extrait le caractère à la position d'indice 1 de la variable **language** et l'affecte à la variable **letter**.
- Le premier caractère est à l'indice 0.
- L'expression entre crochets est appelée un indice. L'indice indique quel caractère vous voulez dans la séquence.

p	y	t	h	o	n
[0]	[1]	[2]	[3]	[4]	[5]

- Vous pouvez utiliser n'importe quelle expression, y compris des variables et des opérateurs, comme indice, mais la valeur de l'indice doit être un entier.



STRINGS

- Pour obtenir la dernière lettre d'une chaîne, vous pourriez être tenté d'essayer quelque chose comme ceci :

```
length = len(language)
```

```
last_character = language[length] → IndexError: string index out of range
```

- Pour obtenir le dernier caractère, vous devez soustraire 1 de **length**:

```
last_character = language[length-1]
```

- Alternativement, vous pouvez utiliser des indices négatifs, qui comptent à partir de la fin de votre string. L'expression **language[-1]** retourne le dernier caractère, **language[-2]** désignera l'avant-dernier, et ainsi de suite.



STRINGS

- Les chaînes dans Python sont intrinsèquement itérables, ce qui signifie que l'itération sur une chaîne donne chaque caractère en sortie.

```
for char in language:  
    print(char)
```

- Utilisez la boucle while pour boucler sur une string en Python

```
index = 0  
while index < len(language):  
    letter = language [index]  
    print(letter)  
    index = index + 1
```

- **Exercice :** Écrivez une fonction qui prend une chaîne de caractère comme argument et affiche ses caractères à l'envers.



STRINGS

- La fonction **enumerate()**: ajoute un compteur à un itérable. Elle est utilisée pour garder un compte du nombre d'itérations effectuées dans la boucle.

```
for i, j in enumerate("string"):
    print(i, j)
```

- La fonction **enumerate()** prend deux paramètres:
 - **iterable** : une séquence, un itérateur ou des objets qui supportent l'itération.
 - **start (facultatif)** : enumerate() commence à compter à partir de ce nombre. Si start est négligé, 0 est pris comme start.



STRINGS

- Le terme anglais de **slice** est associé à l'idée de **découpage** (une part de gâteau ou de pizza).
- En programmation, et en Python en particulier, les slices sont des expressions du langage Python qui vous permettent en une ligne de code d'extraire des éléments d'une liste ou un segment d'une chaîne.
- La sélection d'une slice est similaire à la sélection d'un caractère:

```
comedy = 'Monty Python'  
print(comedy[0:5])
```
- L'opérateur **[n:m]** renvoie la partie de la chaîne du "**n-ème**" caractère au "**m-ème**" caractère, y compris le premier mais excluant le dernier.
- `comedy[0:5]` est une slice et désigne la chaîne extraite de la chaîne **comedy** dont les éléments sont situés de l'indice 0 jusqu'à l'indice 4 inclu.



STRINGS

- Si vous omettez le premier indice (avant les deux-points), la slice commence au début de la string. Si vous omettez le deuxième indice, la slice va à la fin de la string:

`comedy[:5]` → 'Monty'

`comedy[5:]` → 'Python'

- Si le premier indice est supérieur ou égal au second, le résultat est une chaîne vide, représenté par deux guillemets:

`comedy[3:3]` → ''

- Quelle est le resultat de `comedy[:]` ?
- Supposons qu'on veut savoir si un fichier est un fichier python (donc si son nom se termine par l'extension **.py**)?



STRINGS — OBJETS NON MODIFIABLES

- Une fois une chaîne de caractères définie, vous ne pouvez plus modifier un de ses éléments. Le cas échéant, Python renvoie un message d'erreur:

```
greeting = 'Hello, world!'
```

```
greeting[0] = 'J'
```

`TypeError: 'str' object does not support item assignment`

- Par conséquent, si vous voulez modifier une chaîne de caractères, vous devez en construire une nouvelle. Pour cela, n'oubliez pas que les opérateurs de concaténation (+) et de duplication (*) peuvent vous aider.

```
greeting = 'Hello, world!'
```

```
new_greeting = 'J' + greeting[1:]
```

- Dans cet exemple on concatène une nouvelle lettre avec une slice de la chaîne **greeting** et cela n'a pas d'effet sur la chaîne d'origine



STRINGS — METHODES

- Les chaînes sont un exemple d'objets Python. Un objet contient à la fois des données (la chaîne elle-même) et les méthodes, qui sont en fait des fonctions intégrées à l'objet et sont disponibles pour toute instance de l'objet.
- Voici quelques **méthodes** spécifiques aux objets de type **str**:
 - **.upper(), .lower()**: renvoient un texte en majuscule et en minuscule

```
animal = "girafe"
animal.upper()
```
 - **.capitalize()**: Pour mettre en majuscule la première lettre seulement.

```
animal.capitalize()
```



STRINGS — METHODES

- **.split()**: divise une chaîne en une liste en utilisant comme séparateur (par défaut) un (ou des) espace(s) blanc. Il est possible de modifier le séparateur de champs en passant le séparateur comme argument: Il est également intéressant d'indiquer à **.split()** le nombre de fois qu'on souhaite découper la chaîne de caractères avec l'argument **maxsplit**: **.split(maxsplit=1)**.

```
instruments = "violin viola bass"  
instruments.split()
```



Toutes les méthodes de chaîne renvoient de nouvelles valeurs. Ils ne changent pas la chaîne d'origine.



STRINGS — METHODES

- **.find(ch):** recherche une chaîne de caractères passée en argument. Si l'élément recherché est trouvé, alors l'indice du début de l'élément dans la chaîne de caractères est renvoyé. Si l'élément n'est pas trouvé, alors la valeur -1 est renvoyée. Si l'élément recherché est trouvé plusieurs fois, seul l'indice de la première occurrence est renvoyé.

```
animaux = "girafe tigre"  
animaux.find("i")  
animal.find("afe")  
  
animaux.find("i", 5)  
animaux.find("i", 2, 8)
```

- **.replace(ch1,ch2):** substitue une chaîne de caractères par une autre
- ```
animaux = "girafe tigre"
animaux.replace("tigre", "singe")
animaux.replace("i", "o")
```



# STRINGS — METHODES

- **.count()**: compte le nombre d'occurrences d'une chaîne de caractères passée en argument.

```
animaux = "girafe tigre"
animaux.count("i")
animaux.count("tigre")
```

- **.startswith()**: vérifie si une chaîne de caractères commence par une autre chaîne de caractères.

```
line = 'Have a nice day'
line.startswith("Have")
```

- **.strip()**: permet de « nettoyer les bords » d'une chaîne de caractères.

```
chaîne = " \tfonctionne avec les tabulations et les retours à la ligne\n"
chaîne.strip()
```



# STRINGS — METHODES

- **.format()**: formate une chaîne. La chaîne sur laquelle cette méthode est appelée peut contenir du texte littéral ou des emplacements de remplacement délimités par des accolades {}.

```
a, b = 'Spam', 34
print('Les valeurs respectives de a et b sont {} et {}'.format(a, b))
```

Pour spécifier le format dans lequel on veut afficher une variable:

```
g = 0.478040540540540
nb = 10000000000
print('g = {:.0e} et nb = {:.2f}'.format(nb, g))
```

**.2f**: indique qu'on souhaite afficher la variable sous forme d'un float avec deux chiffres après la virgule.

**.0e**: permet d'afficher un nombre en notation scientifique (sous forme de puissance de 10) avec la lettre e



# STRINGS — OPERATEUR DE FORMATAGE %

- On a vu avec les entiers que l'opérateur % ou modulo renvoyait le reste d'une division entière. Cet opérateur existe aussi pour les chaînes de caractères mais il met en place l'écriture formatée.

```
x = 32
nom = "Ali"
print("%s a %d ans" % (nom, x))
```

Le symbole % est d'abord appelé dans la chaîne de caractères (dans l'exemple ci-dessus %d, %s) pour :

- Désigner l'endroit où sera placée la variable dans la chaîne de caractères.
- Préciser le type de variable à formater, **d** pour un **entier** (**i** fonctionne également) ou **s** pour une **string**.

Le signe % est rappelé une seconde fois pour indiquer les variables à formater.



# STRINGS — METHODES

- Python a une fonction appelée **dir** qui liste les méthodes disponibles pour un objet.

```
stuff = 'Hello world'
```

```
dir(stuff)
```

```
['capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format',
'format_map', 'index', 'isalnum', 'isalpha', 'isdecimal', 'isdigit', 'isidentifier', 'islower',
'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower',
'lstrip', 'maketrans', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip',
'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

- Vous pouvez utiliser la fonction **help** pour obtenir des informations sur une méthode.

```
help(str.capitalize)
```

- Meilleure source de documentation:

<https://docs.python.org/library/stdtypes.html#string-methods>



# LISTS

- Comme une string, une liste est une séquence de valeurs. Dans une chaîne, les valeurs sont des caractères; dans une liste, ils peuvent être de n'importe quel type. Les valeurs de la liste sont appelées éléments.
- Il existe plusieurs façons de créer une nouvelle liste; le plus simple est d'enfermer les éléments entre crochets [] :

```
[10, 20, 30, 40]
['Strawberry', 'Kiwi', 'Mango']
```

- Les éléments d'une liste ne doivent pas nécessairement être du même type. La liste suivante contient une chaîne, un flottant, un entier et une autre liste :

```
['spam', 2.0, 5, [10, 20]]
```

- Vous pouvez stocker des listes dans des variables:

```
cheeses, numbers = ['Cheddar', 'Edam', 'Gouda'], [17, 123]
empty = []
empty2 = list()
print(cheeses, numbers, empty)
```



# LISTS

- La syntaxe pour accéder aux éléments d'une liste est la même que pour accéder aux caractères d'une chaîne: l'opérateur crochets.

- L'expression entre crochets spécifie l'indice. Les indices commencent à 0:

```
print(cheeses[0])
```

Cheddar

- La façon la plus courante de parcourir les éléments d'une liste est d'utiliser une boucle **for**. La syntaxe est la même que pour les chaînes:

```
cheeses = ['Cheddar', 'Edam', 'Gouda']
```

```
for cheese in cheeses:
```

```
 print(cheese)
```

- Cela fonctionne bien si vous n'avez besoin que de lire les éléments de la liste. Mais vous voulez mettre à jour les éléments, vous aurez besoin des indices:

```
numbers = [10, 20, 30, 40]
```

```
for i in range(len(numbers)):
```

```
 numbers[i] = numbers[i] * 2
```



# LISTS - OBJETS MODIFIABLES

- Contrairement aux chaînes, les listes sont modifiables car vous pouvez modifier l'ordre des éléments dans une liste ou réaffecter un de ces éléments.  

```
numbers = [17, 123]
numbers[1] = 5
```
  - Les indices de liste fonctionnent de la même manière que les indices de chaînes:
    - ✓ Vous pouvez utiliser n'importe quelle expression, y compris des variables, des opérateurs et des fonctions, comme indice, mais la valeur de l'indice doit être un entier.
    - ✓ Si vous essayez d'accéder à un élément qui n'existe pas, vous obtenez `IndexError`.
    - ✓ Si un indice a une valeur négative, il compte à partir de la fin de la liste.
- i** Étant donné que les listes sont modifiables, il est souvent utile d'en faire une copie avant d'effectuer des opérations!



# LISTS - OPERATEURS

- L'opérateur **in** est utilisé pour vérifier si une valeur est présente dans une liste (ou séquence)

```
cheeses = ['Cheddar', 'Edam', 'Gouda']
print('Edam' in cheeses)
```

- L'opérateur **+** concatène des listes :

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b
```

- De même, l'opérateur **\*** répète une liste un certain nombre de fois. L'exemple suivant répète la liste quatre fois:

```
zeros_list = [0] * 4
```



# LISTS - SLICES

- L'opérateur slice **[n:m]** fonctionne également sur les listes. L'opérateur **[n:m]** renvoie la partie de la liste du "**n-ème**" élément au "**m-ème** élément", y compris le premier mais excluant le dernier.

```
t = ['a', 'b', 'c', 'd', 'e', 'f']
print(t[1:3]) # Affiche: ['b', 'c']
```

- L'opérateur **[debut:fin:pas]** avec trois paramètres, est utilisé pour créer une sous-liste à partir d'une liste existante en spécifiant un début, une fin et un pas.

```
t = ['a', 'b', 'c', 'd', 'e', 'f']
print(t[::2]) # Affiche: ['a', 'c', 'e']
```

- Un opérateur de slice sur le côté gauche d'une affectation peut mettre à jour plusieurs éléments :

```
t = ['a', 'b', 'c', 'd', 'e', 'f']
t[1:3] = ['x', 'y']
print(t) # Affiche: ['a', 'x', 'y', 'd', 'e', 'f']
```



# LISTS - METHODES

- Comme pour les chaînes de caractères, les listes possèdent de nombreuses **méthodes** qui leur sont propres et qui peuvent se révéler très pratiques.

- **.append()**: ajoute un élément à la fin d'une liste.

```
a = [1, 2, 3]
a.append(5) # ou a = a + [5]
```

- **.insert()**: insère un objet dans une liste à un indice déterminé.

```
a = [1, 2, 3]
a.insert(2, -15)
```

- **.remove()**: supprime un élément d'une liste à partir de sa valeur.

```
a = [1, 2, 3]
a.remove(3)
```

- **del**: supprime un élément d'une liste à un indice déterminé. C'est une instruction générale de Python, utilisable pour d'autres objets que des listes.

```
del a[1]
```



# LISTS - METHODES

- **.sort()**: trie les éléments d'une liste du plus petit au plus grand. L'argument **reverse=True** spécifie le tri inverse, c'est-à-dire du plus grand au plus petit élément.

```
a = [3, 1, 2]
a.sort()
a.sort(reverse=True)
```

- **sorted()**: La fonction `sorted()` trie également une liste. Contrairement à la méthode précédente **.sort()**, cette fonction renvoie la liste triée et ne modifie pas la liste initiale. Cette fonction supporte aussi l'argument **reverse=True**.

```
a = [3, 1, 2]
b = sorted(a, reverse=True)
print(a)
print(b)
```



# LISTS - METHODES

- **.reverse()**: inverse une liste.

```
a = [3, 1, 2]
a.reverse()
```

- **.count()**: compte le nombre d'éléments (passés en argument) dans une liste.

```
a = [1, 2, 4, 3, 1, 1]
a.count(1)
```

- **.extend(iter)**: ajoute tous les éléments de l'argument iterable « iter » à une liste existante.

```
t1 = ['a', 'b', 'c']
t2 = ['d', 'e']
t1.extend(t2)
```

- **.index(valeur, [début, [fin]])**: trouve l'indice d'un élément spécifique dans une liste

```
liste = [10, 20, 30, 40, 30]
print(liste.index(30)) # Affiche: 2
```



# PARTICULARITÉS DES MÉTHODES ASSOCIÉES AUX LISTES

- De nombreuses méthodes mentionnées précédemment (`.append()`, `.sort()`, `.reverse()`, etc.) modifient la liste mais ne renvoient rien, c'est-à-dire qu'elles ne renvoient pas d'objet récupérable dans une variable.
- Même si `var = liste.reverse()` est une instruction Python valide, elle n'a aucun intérêt, préférez-lui `liste.reverse()`.
- Certaines méthodes ou instructions des listes décalent les indices d'une liste (par exemple `.insert()`, `del`, etc.).
- Enfin, pour obtenir une liste exhaustive des méthodes disponibles pour les listes, utilisez la fonction `dir(ma_liste)` (`ma_liste` étant une liste).



# LISTES ET FONCTIONS INTÉGRÉES

- Il existe un certain nombre de fonctions intégrées qui peuvent être utilisées sur des listes qui vous permettent d'effectuer des opérations sans écrire vos propres boucles:

```
nums = [3, 41, 12, 9, 74, 15]
print(len(nums))
print(max(nums))
print(min(nums))
print(sum(nums))
```

- La fonction **sum()** ne fonctionne que lorsque les éléments de la liste sont des nombres. Les autres fonctions (**max()**, **len()**, etc.) fonctionnent avec des listes de chaînes et d'autres types qui peuvent être comparables.



# LISTES ET FONCTIONS INTEGRÉES

- Réécrire le programme qui calcule la moyenne d'une liste de nombres saisie par l'utilisateur à l'aide d'une liste.

```
total = 0
count = 0
while (True):
 inp = input('Enter a number: ')
 if inp == 'done':
 break
 try:
 value = float(inp)
 except:
 print('Invalid input')
 continue
 total = total + value
 count = count + 1
average = total / count
print('Average:', average)
```



# LISTES ET FONCTIONS INTEGRÉES

- Réécrire le programme qui calcule la moyenne d'une liste de nombres saisie par l'utilisateur à l'aide d'une liste.

```
numbers = []
while True:
 inp = input('Enter a number: ')
 if inp == 'done':
 break
 try:
 value = float(inp)
 except ValueError:
 print('Invalid input')
 else:
 numbers.append(value)

if numbers:
 average = sum(numbers) / len(numbers)
 print('Average:', average)
else:
 print('No numbers entered.')
```



# LISTES ET OBJETS ITÉRABLES

- Si l'argument passé à la fonction **list()** est une séquence ou un objet itérable le résultat de est une liste avec les éléments de la séquence.

Exemple 1:

```
seq = "CAAAGGTAACGC"
char_list = list(seq)
print(char_list)
```

Exemple 2:

```
seq = range(10)
indice_list = list(seq)
print(indice_list)
```



# TUPLES

- Un tuple est une séquence de valeurs un peu comme une liste.
- Les tuples sont des objets itérables, ordonnés et indexables.
- Les valeurs stockées dans un tuple peuvent être de n'importe quel type.
- La différence importante est qu'un tuple ne peut pas être modifié une fois qu'il est créé. On dit qu'un tuple n'est pas modifiable ou n'est pas mutable (immutable objects)
- Syntaxiquement, un tuple est une suite de valeurs séparées par des virgules :

```
t = 'a', 'b', 'c', 'd', 'e'
```

- Mais il est courant de mettre les tuples entre **parenthèses** pour nous aider à les identifier rapidement:

```
t = ('a', 'b', 'c', 'd', 'e')
```



# TUPLES

- Pour créer un tuple vide:

```
t1 = ()
t2 = tuple()
```

- Si l'argument passé à **tuple()** est une séquence (chaîne, liste, tuple ..), le résultat est un tuple avec les éléments de la séquence:

```
t = tuple('lapin')
```

- Pour créer un tuple avec un seul élément, vous devez inclure la virgule à la fin:

```
t3 = ('a',)
```

- Les opérateurs de comparaison fonctionnent avec des tuples et d'autres séquences. Python commence par comparer le premier élément de chaque séquence. S'ils sont égaux, il passe au élément suivant jusqu'à ce qu'il trouve des éléments qui diffèrent.

```
(0, 1, 2000000) < (0, 3, 4)
```



# TUPLES

- La plupart des opérateurs de liste fonctionnent également sur les tuples.

- L'opérateur `[]` indexe un élément:

```
t = ('a', 'b', 'c', 'd', 'e')
print(t[0])
print(t[-1])
```

- L'opérateur slice `[n:m]` permet d'extraire une partie d'éléments d'un tuple:

```
print(t[1:3])
```

- Si vous essayez de modifier un des éléments du tuple, vous obtenez une erreur:

```
t[0] = 'A'
```

- Mais vous pouvez remplacer un tuple par un autre:

```
t = ('A',) + t[1:]
print(t)
```



# TUPLES

- la fonction **len()** accepte un tuple en argument et retourne le nombre d'éléments qu'il contient.
- l'opérateur **+** permet de joindre des tuples ensemble
- l'opérateur **\*** peut multiplier les tuples, tout comme les listes;
- l'opérateur **=** peut affecter à un tuple un autre tuple
- les opérateurs **in** et **not in** fonctionnent de la même manière que dans les listes.

```
myTuple = (1, 10, 100)
t1 = myTuple + (1000, 10000)
t2 = myTuple * 3
print(len(t2))
print(t1)
print(t2)
print(10 in myTuple)
print(-10 not in myTuple)
```



# TUPLES

- Un tuple permet une affectation multiple:

```
m = ('have', 'fun')
```

```
(x, y) = m
```

```
x, y = y, x
```

```
print(x,y)
```

```
addr = 'monty@python.org'
```

```
uname, domain = addr.split('@')
```

- Il permet également de renvoyer plusieurs valeurs lors d'un appel d'une fonction:

```
def nom_prenom():
```

```
 return "Benhar", "Houda"
```

```
x, y = nom_prenom()
```

- On utilisera un tuple pour définir des sortes de constantes qui n'ont donc pas vocation à changer.



# TUPLES

- Similairement aux autres objets itérables vous pouvez utiliser des boucles pour parcourir les éléments d'un tuple:

```
t = tuple(range(10))
print(t)
for num in t:
 print(num)
```

- **enumerate()** permettait d'itérer **en même temps** sur les indices et les éléments d'un objet itérable:

```
for element in enumerate("python"):
 print(element, type(element))
```

```
for index, value in enumerate("python"):
 print(index, value)
```



# TUPLES

- En Python, utilisez la fonction intégrée **sorted()** pour trier un Tuple. Le tuple doit être passé en argument à la fonction **sorted()**. Les éléments du tuple sont triés (par défaut) par ordre croissant dans la liste renvoyée par la fonction. Vous pouvez utiliser **tuple()** pour convertir ce type de données de liste en un tuple.

```
tuple_ = (5, 2, 24, 3, 1, 6, 7)
sorted_ = tuple(sorted(tuple_))
print('Sorted Tuple:', sorted_)
print(type(sorted_))
```



# TUPLES - EXERCICE

- Écrire une fonction qui prend un en argument un tuple, supprime son dernier élément et retourne le nouveau tuple.
- **Correction:**

```
def del_last_element(my_tuple):
 new_tuple = list(my_tuple)
 del new_tuple[-1]
 return new_tuple
languages = ('Javascript', 'PHP', 'Python', 'PL/SQL')
new = del_last_element(languages)
print(new)
```



# DICTIONNAIRES

- Un dictionnaire en python est une sorte de liste mais au lieu d'utiliser des indices, on utilise des clés alphanumériques.
- Contrairement aux listes qui sont délimitées par des crochets, on utilise des **accolades** pour les dictionnaires.
- Pour initialiser un **dictionnaire** , on utilise la syntaxe suivante:

```
eng2sp = {}
```

**ou**

```
eng2sp = dict()
```

- Vous pouvez considérer un dictionnaire comme une correspondance entre un ensemble d'indices (qui sont appelées clés) et un ensemble de valeurs. L'association d'une clé et une valeur est appelée une paire clé-valeur ou parfois un élément.



# DICTIONNAIRES

- À titre d'exemple, nous allons créer un dictionnaire qui mappe des mots anglais vers des mots espagnols, donc les clés et les valeurs sont toutes des chaînes.
- Pour ajouter des valeurs à un **dictionnaire** il faut indiquer une clé ainsi qu'une valeur:

```
eng2sp = {}
eng2sp['one'] = 'uno'
eng2sp['two'] = 'dos'
print(eng2sp)
```

- On peut aussi initialiser toutes les clés et les valeurs d'un dictionnaire en une seule opération:

```
eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
```

Ou

```
eng2sp = dict(one = 'uno', two = 'dos', three = 'tres')
```

Ou

```
eng2sp = dict([('one', 'uno'), ('two', 'dos'), ('three', 'tres')])
```



# DICTIONNAIRES

- On accède à une **valeur** du dictionnaire en utilisant la **clé** entourée par des crochets avec la syntaxe suivante:

```
print(eng2sp['one'])
```

- Ou en utilisant la fonction **get(key, default)**:

```
print(eng2sp.get("two"))
```

- La fonction **len()** renvoie le nombre de paires clé-valeur :

```
print(len(eng2sp))
```

- L'opérateur **in** vous indique si quelque chose apparaît comme une clé dans le dictionnaire:

```
print('one' in eng2sp)
```

```
print('uno' in eng2sp)
```



# DICTIONNAIRES

- Pour copier un dictionnaire, utilisez la méthode **.copy()** :

```
dict = {"cat" : "chat", "dog" : "chien", "horse" : "cheval"}
copyDict = dict.copy()
```

- La méthode **.update()** permet de fusionner deux dictionnaires .

```
dict1 = {"cat": "chat", "dog": "chien", "horse": "cheval"}
dict2 = {"duck": "canard"}
dict1.update(dict2)
print(dict1)
print(dict2)
```



# DICTIONNAIRES

- Il est possible de supprimer une entrée en indiquant sa clé, comme pour les listes:

```
del eng2sp["one"]
```

- Pour supprimer tous les éléments du dictionnaire, vous devez utiliser la méthode **.clear()**:

```
eng2sp.clear()
```

- Si on souhaite voir toutes les associations clés / valeurs, on peut itérer sur un dictionnaire de la manière suivante:

```
nombre_de_roues = {"voiture": 4, "vélo": 2, "tricycle": 3}
```

```
for cle in nombre_de_roues:
```

```
 print(cle, nombre_de_roues[cle])
```

- Les méthodes **.keys()**, **.values()**, renvoient, comme vous vous en doutez, les clés et les valeurs d'un dictionnaire:

```
print(nombre_de_roues.keys())
```

```
print(nombre_de_roues.values())
```



# DICTIONNAIRES, LISTES ET TUPLES

- Les dictionnaires ont une méthode **.items()** qui renvoie une liste de tuples, où chaque tuple est une paire clé-valeur:

```
d = {'a':10, 'b':1, 'c':22}
t = list(d.items())
t.sort()
print(t)
```

- La fonction **zip()** nous permet d'initialiser un dictionnaire avec deux listes, une pour les clés et une pour les valeurs

```
languages = ['Javascript', 'PHP', 'Python']
extensions = ['.js', '.php', '.py']
a_dict = dict(zip(languages, extensions))
```

